

02_Module_2_Service_Level_Management

Module 2 — Service Level Management

Duration: 8 Hours

2.1 The Hierarchy

SLA ← The contract with the customer (external, financial penalties)
└─ SLO ← The target you set internally (stricter than SLA)
 └─ SLI ← What you actually measure (the raw metric)

The safety margin rule: SLO must always be tighter than SLA. Your internal alarm fires before the customer penalty kicks in.

SLA: 99.5% availability
SLO: 99.9% availability ← You wake up here
 ← Gap = 0.4% safety margin
If SLI drops to 99.7% → SLO breached → engineering responds
But SLA (99.5%) is safe → no customer penalty

2.2 Service Level Indicators (SLIs)

Definition

An SLI is a **quantitative measurement** of a specific aspect of service behaviour:

$$SLI = \text{Good Events} / \text{Total Events}$$

SLI Categories for Banking

Category	What It Measures	Banking Example
Availability	Is the service up?	successful ATM withdrawals / total attempts
Latency	Is it fast enough?	transfers completed < 3s / total transfers
Error Rate	Is it producing errors?	successful API calls / total API calls
Throughput	Can it handle the load?	transactions processed per minute
Correctness	Is the output right?	transactions posted correctly / total transactions

SLI Aggregation Strategies

SLIs can be aggregated in different ways depending on the service architecture:

Strategy	How It Works	When To Use
Request-based	Count good vs total requests	User-facing services where each request is independent
Window-based	Measure availability in fixed time windows	Infrastructure services (e.g., “was the database up this minute?”)
Population-based	Measure ratio of good users to total users	When user-level experience matters more than request count
Bucket-based	Measure metrics against defined thresholds	Latency SLIs (p50, p95, p99 within thresholds)

Example — Request-based:

ATM withdrawals this month:

- Total: 100,000
- Successful: 99,990
- SLI = $99,990 / 100,000 = 99.99\%$

Example — Window-based:

ATM service checked every minute:

- Total minutes this month: 43,200
- Minutes with 100% uptime: 43,150
- SLI = $43,150 / 43,200 = 99.88\%$

The choice matters: a 5-minute outage affects request-based less (only the requests during that window) but window-based more (5 whole minutes count as down).

2.3 Service Level Objectives (SLOs)

Setting SLOs

An SLO is the **target value** for your SLI over a defined time window:

SLO = "SLI must stay above X% over the next Y days"

How NOT to set SLOs:

✗ “Set it to whatever the system currently achieves” — this locks in current performance and doesn’t drive improvement.

✗ “Set it to 99.999% because the bank demands it” — this ignores cost and user perception.

How to set SLOs correctly:

1. **Identify user journeys** — map the critical paths users take
2. **Determine satisfaction threshold** — what does “good enough” look like from the user’s perspective?
3. **Evaluate current capability** — what can the system actually deliver?
4. **Negotiate with business** — explain the cost of tighter SLOs
5. **Start conservative** — you can always tighten later

Sample SLOs for Union Bank

Service	SLI	SLO	Window	Budget
ATM withdrawals	Availability	99.9%	30 days	43m
Mobile app login	Latency (< 2s)	95%	30 days	N/A (latency)
Online transfers	Correctness	99.99%	30 days	4m 19s
Internet banking	Availability	99.95%	30 days	21m 36s
Core banking system	Availability	99.995%	30 days	2m 10s

2.4 Service Level Agreements (SLAs)

An SLA is a **formal contract** with financial or legal consequences for breach.

	SLO	SLA
Audience	Internal engineering team	Customer + legal + regulators
Consequence of miss	Feature freeze, engineering focus	Financial penalty, regulatory action
Tightness	Stricter (0.1–0.5% margin)	Looser
Changes	Can be adjusted internally	Requires legal review

Banking SLA examples:

- “If ATM availability drops below 99.5% in any calendar month, all ATM fees waived for that month.”
 - “If online transfer processing exceeds 10 seconds, the transaction fee is refunded.”
 - “If internet banking is unavailable for > 4 hours in a single day, affected business customers receive service credits.”
-

2.5 Error Budgets

Calculation

Error Budget = $(100\% - \text{SLO}\%) \times \text{Total Time in Window}$

Worked example:

Service: ATM Network

SLO: 99.9%

Window: 30 days

Budget = $(1.000 - 0.999) \times (30 \times 24 \times 60)$
 = $0.001 \times 43,200$
 = 43.2 minutes per month

Budget Consumption Tracking

Each incident consumes from the budget. Track consumption throughout the window:

Monthly budget: 43.2 minutes

Week 1: Database restart – 8 min budget consumed → 35.2 remaining

Week 2: Network issue – 12 min budget consumed → 23.2 remaining

Week 3: Bad deployment – 30 min budget consumed → -6.8 → EXHAUSTED

Composite SLOs and Budgets

When a service depends on multiple sub-services, the overall error budget is a composite:

Mobile App depends on:

- Auth Service (SLO 99.99%)
- Core Banking API (SLO 99.95%)
- SMS Gateway (SLO 99.9%)

Composite SLO = product of sub-service SLOs (if serial dependencies)
 $= 0.9999 \times 0.9995 \times 0.999 = 0.9984 = 99.84\%$

Monthly budget for mobile app: $(1 - 0.9984) \times 43,200 = \sim 69$ minutes

2.6 SLO-Based Alerting with Burn Rates

Why Burn Rate Alerting Replaces Static Thresholds

Static threshold alerts (e.g., “alert if error rate > 5%”) have fundamental problems:

1. They don’t account for the SLO — 5% errors might be fine for some services and catastrophic for others
2. They trigger on momentary spikes that don’t affect the SLO
3. They miss slow-burn degradations that stay just below the threshold

Multi-Window Burn Rate Approach

Google SRE recommends four alerting windows to cover different burn scenarios:

Window	Burn Rate Threshold	Severity	Purpose
1 hour	≥ 2	Critical (page)	Fast-burning, catastrophic
6 hours	≥ 1	Critical (page)	Will exhaust budget
3 days	≥ 0.5	Warning (ticket)	Chronic degradation
30 days	≥ 0.1	Warning (ticket)	Slow-burn trend

Algorithm: 1. For each window, calculate actual SLI over the window 2. Divide by target SLI (1 — SLO) 3. Compare against threshold

Implementation in Prometheus:

```
groups:
- name: slo_alerts
  rules:
    # 1-hour window, burn rate  $\geq 2$ 
    - alert: SLOFastBurn
      expr: |
        (
          1 - (sum(rate(http_requests_total{status!~"2.."}[1h]))
              / sum(rate(http_requests_total[1h])))
        ) / (1 - 0.999) > 2 # SLO = 99.9%
      for: 2m
      labels:
        severity: critical
        window: 1h
```

```

# 6-hour window, burn rate ≥ 1
- alert: SLOExhaustion
  expr: |
    (
      1 - (sum(rate(http_requests_total{status!~"2.."}[6h]))
        / sum(rate(http_requests_total[6h])))
    ) / (1 - 0.999) > 1
  for: 5m
  labels:
    severity: critical
    window: 6h

# 3-day window, burn rate ≥ 0.5
- alert: SLOChronicBurn
  expr: |
    (
      1 - (sum(rate(http_requests_total{status!~"2.."}[3d]))
        / sum(rate(http_requests_total[3d])))
    ) / (1 - 0.999) > 0.5
  for: 15m
  labels:
    severity: warning
    window: 3d

```

Choosing Alerting Windows

The windows and thresholds must account for your on-call response time:

- If your on-call engineer takes 15 minutes to respond: the 1-hour window (≥ 2 burn rate) gives you ~30 minutes of budget to operate before exhausting
- The 6-hour window (≥ 1 burn rate) handles slower events like traffic shifts or partial failures
- The 3-day window catches gradual degradation (e.g., memory leaks, slow data corruption)

2.7 Error Budget Policy

An error budget policy defines **exactly what happens** at each consumption level.

Sample Policy

Service: Union Bank Mobile App
 SLO: 99.95% (21m 36s monthly budget)

GREEN (> 50% remaining: > 10.8 min):

- Normal deployment pace
- Any change can be deployed
- Risk-taking encouraged

YELLOW (20–50% remaining: 4.3–10.8 min):

- Deployments require peer review
- High-risk changes need SRE lead approval
- Team focuses 30% on reliability work

RED (0–20% remaining: < 4.3 min):

- No new feature deployments
- All engineering effort on reliability
- Daily incident review meetings

- SRE lead reviews every deployment decision

EXHAUSTED (0% remaining):

- Complete feature freeze until next month
- Mandatory post-mortem for every incident this period
- Engineering director must approve any exception
- On-call rotation doubled

2.8 Classroom Activity: Build SLOs with Burn Rate Alerts

Time: 2 hours

Part 1 — Individual (30 min)

Pick one Union Bank service and define:

1. **Three SLIs** — write the formula for each, specify request-based or window-based
2. **SLO targets** — justify why you chose each number
3. **Error budget** — calculate monthly budget in minutes
4. **SLA** — set an SLA 0.1-0.5% looser than the SLO
5. **Three burn rate alerts** — specify window, burn rate threshold, severity, and victim

Part 2 — Group Scenario (45 min)

Mobile banking app – monthly SLO: 99.95% (21.6 min budget)

Incidents this month:

- Day 3: App crashed 15 min (connection pool)
- Day 11: Login failures 22 min (SSO token expiry)
- Day 19: Transfer service slow 40 min (not down, latency > 5s)
- Day 27: Full outage 18 min (database failover)

Questions: 1. How much error budget was consumed by each incident? (Only full downtime counts against availability SLO — the latency issue is a separate latency SLO) 2. At what point was the availability budget exhausted? 3. At what point should burn rate alerts have fired? (1-hour window ≥ 2 , 6-hour window ≥ 1) 4. What should the team have done after the Day 11 alert? 5. Did the bank breach its SLA? (SLA = 99.9%) 6. Write a 5-point error budget policy for this service

Part 3 — Presentations (45 min)

Each group presents. Class challenges assumptions about SLO tightness and alerting windows.

Key Terms

Term	Definition
SLI	Service Level Indicator — the measured value (e.g., 99.5% availability)
SLO	Service Level Objective — the internal target (e.g., $\geq 99.9\%$)
SLA	Service Level Agreement — the customer contract with penalties
Error Budget	100% — SLO%; the allowed unreliability in a window
Burn Rate	Rate of error budget consumption relative to the window

Term	Definition
Multi-window alerting	Alerting on multiple time windows to catch fast and slow burns
Composite SLO	Combined SLO of dependent services (product of individual SLOs)
Safety Margin	Gap between SLO and SLA (~0.1–0.5%)

Further Reading

- [Google SRE Workbook, Chapter 2 — Implementing SLOs](#)
- [Google SRE Workbook, Chapter 5 — Alerting on SLOs](#)
- [Burn Rate Alerting Deep Dive — Google CRE](#)
- [SLOs for the Rest of Us — Honeycomb](#)